



FINAL PROJECT REPORT

GA-Based Network Routing Optimization Model in MATLAB

TABLE OF CONTENTS

1. Introduction	2
2. Problem Definition and System Model	2
3. GA-Based Network Optimization Approach	2
(A) main.m	3
(B) fitness_routing.m	3
(C) visualize_results.m	3
Figure 1: MATLAB implementation of the GA-based network optimization model including the main script and customized fitness function (latest version available on GitHub)	3
4. Results and Technical Interpretation	4
Figure 2: Command window output showing the optimized routing path and final fitness value obtained by the genetic algorithm	4
Figure 3: Convergence plot illustrating the evolution of the best and mean fitness values over generations during the genetic algorithm optimization process (view in GitHub)	4
Figure 4: Optimized routing path over the network topology obtained using the genetic algorithm (view in GitHub)	5
4.1.Lookup Table	6
Figure 5: Network lookup information including adjacency matrix, capacity matrix and node traffic tables used by the genetic algorithm for routing evaluation	6
Figure 6: Optimal routing path and corresponding convergence behavior obtained by the genetic algorithm under traffic, connectivity and capacity constraints (view in GitHub)	7
5. Conclusion	7
6. Acknowledgment	7



1. Introduction

This report is submitted as the final project for the Genetic Algorithms course. It focuses on the application of genetic algorithms to a computer network optimization problem, specifically routing optimization in a computer network which is implemented in MATLAB. Unlike classical topology design problems, this project addresses the problem of determining efficient communication paths between network nodes. A customized fitness function is designed and a genetic algorithm is employed to identify optimal routing paths under realistic network constraints.

The proposed model incorporates key network characteristics such as traffic load, link connectivity and capacity limitations. The performance of the optimization process is evaluated through program outputs and graphical results including convergence plots and routing visualizations.

2. Problem Definition and System Model

In this project, the problem addressed is the **routing optimization of a computer network** consisting of multiple nodes distributed in a two-dimensional space. Each node represents a computer with fixed coordinates (x and y). Unlike topology optimization, node locations remain constant and the objective is to determine the most efficient routing path.

The optimization focuses on finding the optimal communication route from Node 1 (source) to Node 6 (destination). Each candidate solution represents a sequence of nodes corresponding to a possible routing path. The system model incorporates several realistic network constraints. An adjacency matrix is used to define valid communication links based on a maximum allowable distance of 40 units. A capacity matrix represents link limitations, ensuring that routes using overloaded connections are penalized. In addition, nodes are assigned traffic levels to reflect varying communication demands.

The fitness function evaluates each route by combining distance-based communication cost with traffic weighting, while penalizing invalid connections and capacity violations. This enables the genetic algorithm to identify routing paths that are efficient, feasible and consistent with realistic network behavior. Overall, the problem is formulated as a constraint-based routing optimization problem where the genetic algorithm iteratively improves candidate solutions to converge toward an optimal communication path.

3. GA-Based Network Optimization Approach

In this study, a genetic algorithm is employed to solve a routing optimization problem in a computer network. Each individual in the population represents a candidate routing path between a predefined source node (Node 1) and destination node (Node 6).

The genetic algorithm (MATLAB's built-in genetic algorithm - *ga*) encodes each routing path as a fixed-length chromosome where each gene corresponds to a node index. Since the genetic algorithm operates in a continuous search space, chromosome values are discretized by rounding and post-processed to ensure valid routing paths. Consecutive duplicate nodes are removed and the source and destination nodes are enforced at the beginning and end of each route.

The optimization process follows the standard genetic algorithm workflow. An initial population of random routing paths is generated and each individual is evaluated using a customized fitness function. Based on fitness values, selection, crossover and mutation operators are applied to evolve the population over successive generations. This iterative process allows the algorithm to gradually improve routing solutions and converge toward an optimal path.

<pre> 1 % Computer Network Optimization using Genetic Algorithm 2 % Goal: Optimize the physical placement of computers (nodes) in a 2D space using a genetic algorithm. 3 % Decision variables: x and y coordinates of each node 4 % Overall idea: GA searches for coordinate configurations that minimize communication cost while satisfying network constraints. 5 clc; clear; close all; 6 7 % Output directory for saving figures 8 outputDir = './outputs'; 9 10 if ~exist(outputDir, 'dir') 11 mkdir(outputDir); 12 end 13 14 % Problem Parameters 15 nNodes = 10; % Number of network nodes 16 routeLength = 8; % max nodes in route 17 nvars = routeLength; % decision variables = route nodes 18 19 % Genetic Algorithm Configuration 20 options = gaoptimset(... 21 'PopulationSize', 50, ... % Number of individuals 22 'Generations', 200, ... % Number of iterations 23 'Display', 'iter', ... % Show progress in command window 24 'PlotFcn', @gaplotbestf); % Plot best fitness per generation 25 26 % Run Genetic Algorithm 27 % Calls custom fitness function to evaluate solutions 28 coords = rand(nNodes, 2) * 100; % fixed node positions 29 30 % Adjacency matrix (connectivity graph) 31 adjMatrix = zeros(nNodes); 32 maxDist = 40; 33 34 for i = 1:nNodes 35 for j = 1:nNodes 36 if i < j 37 d = norm(coords(i,:) - coords(j,:)); 38 if d <= maxDist 39 adjMatrix(i,j) = 1; 40 end 41 end 42 end 43 end 44 45 % Link capacity matrix (random capacities) 46 capacityMatrix = randi([1 10], nNodes, nNodes); 47 48 traffic = randi([1 5], nNodes, 1); 49 50 fitnessFcn = @(x) fitness_routing(x, coords, traffic, adjMatrix, capacityMatrix); 51 [x_best, fval] = ga(fitnessFcn, nvars, [], [], [], [], [], ... 52 ones(1,nvars), nNodes*ones(1,nvars), [], options); 53 </pre>	<pre> 1 function score = fitness_routing(route, coords, traffic, adjMatrix, capacityMatrix) 2 3 n = size(coords,1); 4 5 % Round to integer 6 route = round(route); 7 8 % Clamp values (extra safety) 9 route(route < 1) = 1; 10 route(route > n) = n; 11 % remove only consecutive duplicates 12 route = route([true diff(route)~=0]); 13 14 % Force start & end 15 route(1) = 1; 16 route(end) = 6; 17 18 totalCost = 0; 19 penalty = 0; 20 21 for i = 1:length(route)-1 22 nodeA = route(i); 23 nodeB = route(i+1); 24 25 % Check connectivity using adjacency matrix 26 if adjMatrix(nodeA, nodeB) == 0 27 penalty = penalty + 150; 28 end 29 30 % Compute distance 31 d = norm(coords(nodeA,:) - coords(nodeB,:)); 32 33 % Traffic-aware cost 34 trafficWeight = (traffic(nodeA) + traffic(nodeB)) / 2; 35 36 % Capacity constraint 37 capacity = capacityMatrix(nodeA, nodeB); 38 39 if capacity < trafficWeight 40 penalty = penalty + 200; % congestion penalty 41 end 42 43 totalCost = totalCost + d * trafficWeight; 44 end 45 46 % Penalize short route 47 if length(route) < 3 48 penalty = penalty + 500; 49 end 50 51 score = totalCost + penalty; 52 53 end </pre>
--	---

(A) main.m

(B) fitness_routing.m

```

1 function visualize_results(x_best, fval, nNodes, outputDir, coords)
2
3 route = round(x_best);
4 route = route([true diff(route)~=0]); % remove repeats
5 route(1) = 1;
6 route(end) = 6;
7
8 figure;
9 scatter(coords(:,1), coords(:,2), 200, 'filled');
10 hold on;
11
12 % node labels
13 for i = 1:nNodes
14     text(coords(i,1)+2, coords(i,2)+2, ['Node ' num2str(i)]);
15 end
16
17 % draw route
18 for i = 1:length(route)-1
19     plot([coords(route(i),1), coords(route(i+1),1)], ...
20         [coords(route(i),2), coords(route(i+1),2)], ...
21         'r-', 'LineWidth', 2);
22 end
23
24 title('Routing Optimization (GA)');
25 grid on;
26
27 % save
28 fileName = sprintf('routing_%dnodes.png', nNodes);
29 exportgraphics(gcf, fullfile(outputDir, fileName), 'Resolution', 300);
30
31 fprintf('\nOptimal Route:\n');
32 disp(route);
33 fprintf('Total Cost: %.2f\n', fval);
34
35 end

```

(C) visualize_results.m

Figure 1: MATLAB implementation of the GA-based network optimization model including the main script and customized fitness function ([latest version available on GitHub](#))

The overall GA structure is implemented in MATLAB and organized into three main components:

(A) main.m: Defines the genetic algorithm parameters, initializes the network model and executes the optimization process.

(B) fitness_routing.m: Evaluates routing paths based on communication cost and network constraints.

(C) visualize_results.m: Visualizes the optimized routing path and convergence behavior.

4. Results and Technical Interpretation

The performance of the proposed genetic algorithm is evaluated through convergence analysis and routing visualization. [Figure 2](#) presents the command window output including the final optimized routing path and the corresponding fitness value.

Single objective optimization: 8 Variables									
Options:									
CreationFcn: @gacreationuniform									
CrossoverFcn: @crossover scattered									
SelectionFcn: @selection stochastic									
MutationFcn: @mutation adaptfeasible									
Generation	Func-count	Best f(x)	Mean f(x)	Stall Generations	Generation	Func-count	Best f(x)	Mean f(x)	Stall Generations
1	100	314.7	1234	0	31	1510	199.2	199.2	29
2	147	199.2	1091	0	32	1557	199.2	199.2	30
3	194	199.2	928.9	1	33	1604	199.2	199.2	31
4	241	199.2	789.8	2	34	1651	199.2	199.2	32
5	288	199.2	650.8	3	35	1698	199.2	199.2	33
6	335	199.2	462	4	36	1745	199.2	199.2	34
7	382	199.2	383.3	5	37	1792	199.2	199.2	35
8	429	199.2	315.7	6	38	1839	199.2	199.2	36
9	476	199.2	265	7	39	1886	199.2	199.2	37
10	523	199.2	241.3	8	40	1933	199.2	199.2	38
11	570	199.2	218.1	9	41	1980	199.2	199.2	39
12	617	199.2	206.6	10	42	2027	199.2	199.2	40
13	664	199.2	201.1	11	43	2074	199.2	199.2	41
14	711	199.2	199.2	12	44	2121	199.2	199.2	42
15	758	199.2	199.2	13	45	2168	199.2	199.2	43
16	805	199.2	199.2	14	46	2215	199.2	199.2	44
17	852	199.2	199.2	15	47	2262	199.2	199.2	45
18	899	199.2	199.2	16	48	2309	199.2	199.2	46
19	946	199.2	199.2	17	49	2356	199.2	199.2	47
20	993	199.2	199.2	18	50	2403	199.2	199.2	48
21	1040	199.2	199.2	19	51	2450	199.2	199.2	49
22	1087	199.2	199.2	20	52	2497	199.2	199.2	50
23	1134	199.2	199.2	21	ga stopped because the average change in the fitness value is less than options.FunctionTolerance.				
24	1181	199.2	199.2	22	Warning: UI components will not be included in the output.				
25	1228	199.2	199.2	23	Optimal Route:				
26	1275	199.2	199.2	24	1 5 9 6				
27	1322	199.2	199.2	25	Total Cost: 199.19				
28	1369	199.2	199.2	26					
29	1416	199.2	199.2	27					
30	1463	199.2	199.2	28					

Figure 2: Command window output showing the optimized routing path and final fitness value obtained by the genetic algorithm

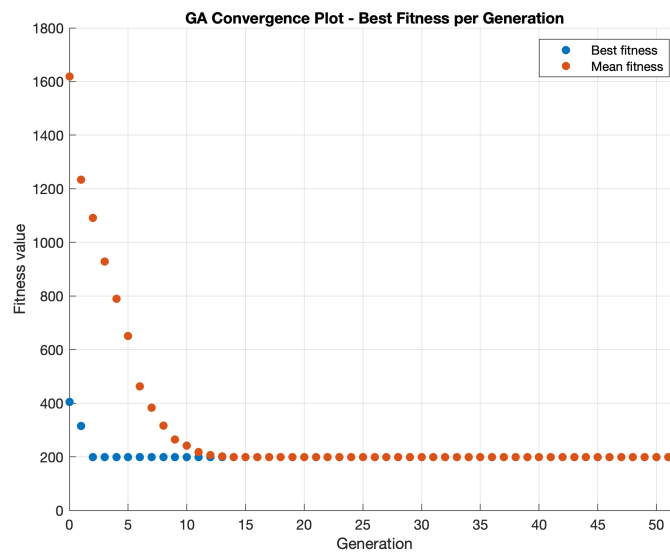


Figure 3: Convergence plot illustrating the evolution of the best and mean fitness values over generations during the genetic algorithm optimization process ([view in GitHub](#))

The convergence behavior of the genetic algorithm is illustrated in [Figure 3](#). The best fitness value decreases rapidly during the initial generations indicating effective exploration of the search space. After approximately 10–15 generations, the fitness value stabilizes showing that the population has converged to an optimal or near-optimal solution.

[Figure 4](#) shows the optimized routing path over the network topology. The selected route demonstrates that the genetic algorithm identifies efficient multi-hop paths by selecting appropriate intermediate nodes rather than relying solely on direct connections. This behavior confirms that the model successfully captures realistic network routing characteristics.

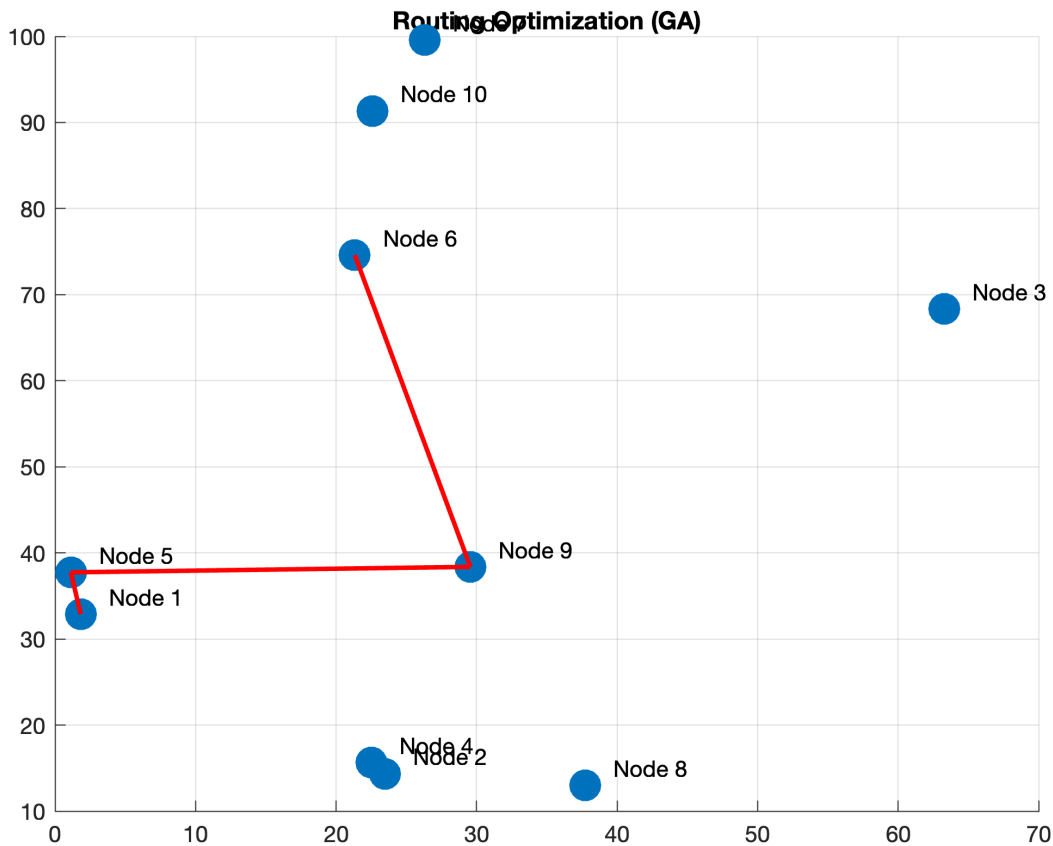


Figure 4: Optimized routing path over the network topology obtained using the genetic algorithm ([view in GitHub](#))

Key Observations:

- The genetic algorithm converges rapidly and consistently to stable routing solutions.
- Intermediate nodes are selected when they reduce overall communication cost under traffic and capacity constraints.
- Different runs may produce different optimal routes due to randomized network configurations; however, the convergence behavior remains consistent.

Overall, the results demonstrate that the proposed GA-based approach effectively solves the constraint-based routing optimization problem and produces realistic and interpretable routing decisions.

4.1.Lookup Table

To improve the interpretability of the model, a lookup table is introduced to explicitly present node traffic, connectivity and link capacities as shown in [Figure 5](#).

The node table reveals that Node 2 and Node 6 have low traffic values, while Node 1 has higher traffic. The adjacency matrix ensures that only physically connected nodes are used and the capacity matrix limits the allowable traffic on each link.

The impact of these parameters can be observed in the final routing result ([Figure 5](#)), where the optimal path is:

$$1 \rightarrow 2 \rightarrow 6$$

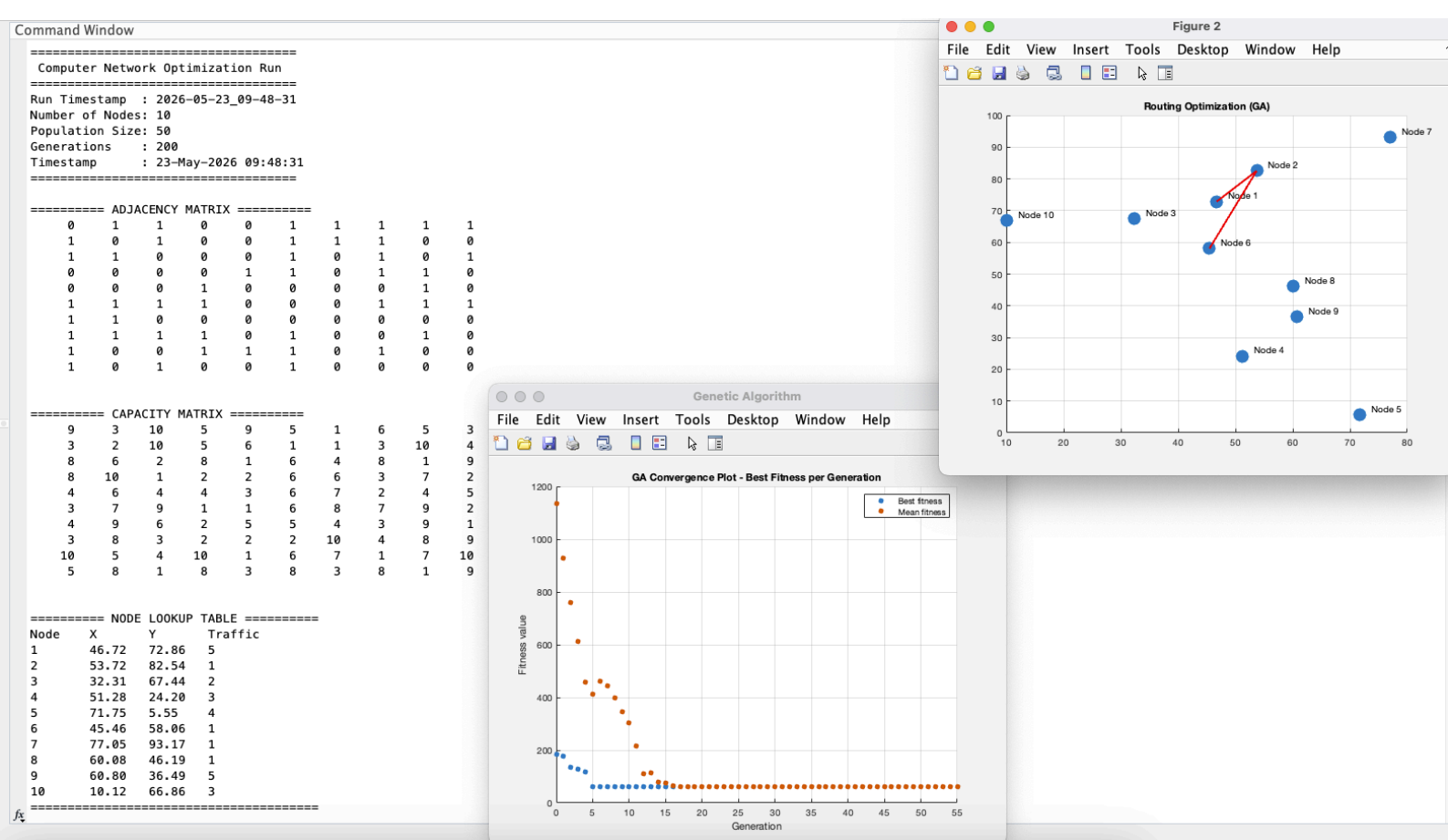


Figure 5: Network lookup information including adjacency matrix, capacity matrix and node traffic tables used by the genetic algorithm for routing evaluation

Although other alternatives such as Node 3 may seem possible, the algorithm prefers Node 2 due to its lower traffic and feasible connections. Additionally, the selected links satisfy capacity constraints avoiding congestion penalties.

Finally, the convergence plot ([Figure 6](#)) shows that the algorithm quickly stabilizes, indicating successful optimization.

Generation	Func-count	Best f(x)	Mean f(x)	Stall Generations
31	1510	61.68	61.68	26
32	1557	61.68	61.68	27
33	1604	61.68	61.68	28
34	1651	61.68	61.68	29
35	1698	61.68	61.68	30
36	1745	61.68	61.68	31
37	1792	61.68	61.68	32
38	1839	61.68	61.68	33
39	1886	61.68	61.68	34
40	1933	61.68	61.68	35
41	1980	61.68	61.68	36
42	2027	61.68	61.68	37
43	2074	61.68	61.68	38
44	2121	61.68	61.68	39
45	2168	61.68	61.68	40
46	2215	61.68	61.68	41
47	2262	61.68	61.68	42
48	2309	61.68	61.68	43
49	2356	61.68	61.68	44
50	2403	61.68	61.68	45
51	2450	61.68	61.68	46
52	2497	61.68	61.68	47
53	2544	61.68	61.68	48
54	2591	61.68	61.68	49
55	2638	61.68	61.68	50

ga stopped because the average change in the fitness value is less
Warning: UI components will not be included in the output.

Optimal Route:
1 2 6

Total Cost: 61.68

--- ROUTE DETAILS ---
Link 1 -> 2 | Dist: 11.95 | Traffic: 3.00 | Capacity: 3
Link 2 -> 6 | Dist: 25.84 | Traffic: 1.00 | Capacity: 1

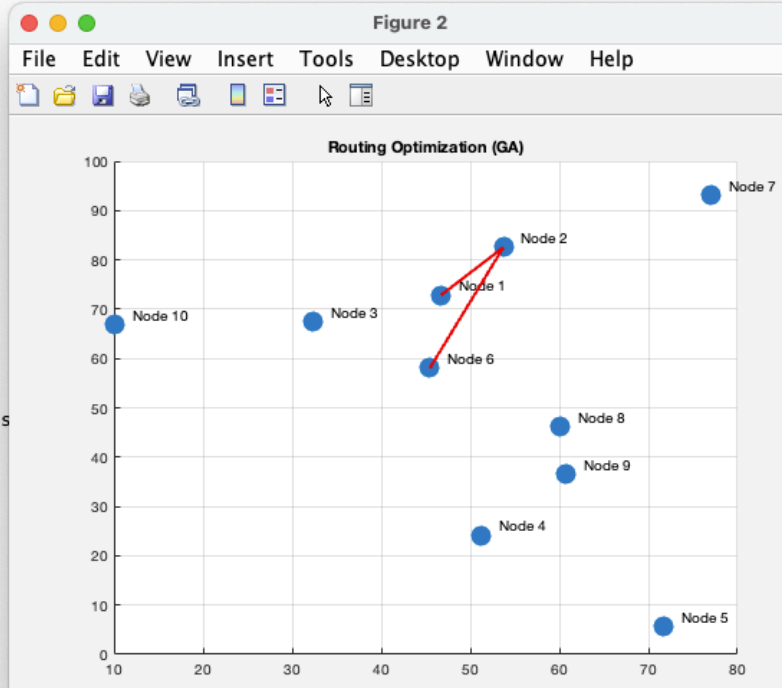


Figure 6: Optimal routing path and corresponding convergence behavior obtained by the genetic algorithm under traffic, connectivity and capacity constraints ([view in GitHub](#))

5. Conclusion

In this final project, genetic algorithms were applied to a computer network optimization problem and implemented in MATLAB. The focus was on **routing optimization** instead of node placement under realistic network constraints. A customized fitness function was designed by incorporating traffic-aware cost, adjacency-based connectivity and link capacity limitations. The results show that the genetic algorithm can effectively identify feasible and efficient routing paths between a source and a destination node. The model produces realistic multi-hop routing decisions by selecting intermediate nodes when necessary. Although results vary due to randomly generated network configurations, the algorithm consistently converges to stable solutions. Overall, the study demonstrates that genetic algorithms are effective for constraint-based network routing optimization.

6. Acknowledgment

The author used Microsoft Copilot as a writing and organization aid. All technical design, implementation, experiments and scientific analysis were carried out by the author and full human judgment was maintained throughout the project. The complete source code and experimental results are available in the project's [GitHub repository](#) released under the MIT License.